CS408 – Progress Report

Digital Logic Simulator

Aims and Objectives

A problem with learning the practical applications of logic is not the quantity of material available, but the lack of a simple, comprehensive guide that takes the student from the basics of digital logic, to simple devices and how they can be connected together to build a basic computer. This project aims to solve this problem by providing a clean and approachable introduction to logic by combining a digital logic simulator and a set of lessons to teach the student how to build a working computer.

The primary goal of this project is to produce a piece of software that acts as a simulator for digital logic circuits. The project's secondary goal is to provide a set of lessons that will teach the user digital logic in a practical and interactive way. The final result of this project will be a tool that acts as an all-in-one practical guide on how to produce a very simple, Turing-complete computer, based on the von Neumann architecture. This tool will be targeted at both students and casual learners looking for an interactive introduction to computational logic. The program should be able to teach someone, who knows nothing or little about digital logic, step-by-step how to build increasingly complex digital devices that will be combined to produce a functioning computer. This will be purely in terms of logic, without requiring the user to learn about electronic engineering during the process.

The logic simulator will provide a simple, grid-based environment in which users can place wires, logic gates, and circuit packages. The circuit packages and logic gates will be similar to semiconductor packages, using pins to represent the inputs and outputs of the packages. Wires will be used to connect these pins to carry signals between them. Wires will be organised by coloured layers, meaning that wires of differing color cannot directly pass signals between each other. This allows for the creation of complex, layered wiring, but with clear visual distinction between different layers.

Each lesson will provide the user with two circuit packages, one will provide a set of pre-defined inputs, the other will take a set of outputs. The purpose of these packages will be to test that the user's circuit works as expected. If the received outputs are correct, the user will be able to progress to the next lesson. Each lesson will also provide a briefing note, outlining the task and provide the user with the information required to complete the task.

The program will also provide a "sandbox" mode in which there is no set objective, and the user is free to experiment with the simulator.

Background Information

Digital logic concerns the use of the binary number system, its combination of arithmetic and logic, and operations used to transform these numbers. Digital logic is the transformation of binary values to produce an output through the prime operators of Boolean algebra (conjunction, disjunction and negation). In the context of this project, digital logic refers to the use of combinatorial and sequential logic to perform boolean algebra on a set of inputs. Combinatorial logic deals with the direct transformation from an input to an output, regardless of the state of the digital logic circuit in a previous pass, as in, it has no memory. Combinatorial logic circuits can be represented as a truth table, as previous states has no effect on the output of the circuit. Sequential logic comes with the additional condition that state can affect the output of the circuit. Simply, this refers to memory, as memory can change state across different inputs on a circuit, but the information held in memory will most likely affect how the circuit calculates its output. In this case, a sequential digital logic circuit cannot be represented as a truth table, as the same input may produce a different output depending on the contents of a connected memory circuit.

Since this project will culminate in the user building a simple computer, it is worth exploring what a computer is in more concise terms. The most well known theoretical model of computation is the Turing machine, which describes an infinitely long tape of memory, subdivided into cells, each holding a “symbol”. The machine has a read/write head that can read a symbol from a cell or write a symbol to a cell. Each symbol represents an operation defined by a finite table of user-specified instructions. The machine would read the symbol at its current position, execute that instruction, write a symbol to the tape based on that instruction, and then advance to the next cell, forming the basis of a fetch, decode, execute cycle. The purpose of the Turing machine is that, given any algorithm, a turing machine can be created to simulate the logic of that algorithm. A machine capable of emulating any turing machine is said to be “Turing-complete”.

A turing machine as it is described here is not practical for computation, but it provides a minimal model for what our machine should be able to achieve. The computer the user will build in the final lesson will be based on the Von Neumann architecture. The Von Neumann architecture is a model for a digital computer that reduces the overall structure of the machine to a few simple components: A central processing unit, a memory unit, an input device and an output device. The central processing unit is comprised of a control unit, an arithmetic logic unit, and registers. The control unit consists of an instruction register and a program counter. The memory unit serves to store data and instructions.

Our machine will be a reduced form of this, in that the input and output devices will, for the most part, be non-existent, aside from using input nodes to manually control the state of the

machine's inputs. However, this is still enough to build a machine capable of emulating a Turing machine, and so it is Turing-complete.

There are many pieces of software that tackle this problem domain, at least in terms of the simulator. Some do this better than others, and some have solved the common issues that are unique to these types of simulators. The first is Boolr, a Dutch high school project implemented in Javascript. The notable attraction of this program is its clean interface and visual style. Despite its sharp, modern look it had several problems. The main issue is with small cyclic circuits, such as flip-flops, which could oscillate under certain circumstances, toggling the flip-flop very rapidly and not being able to settle its state. Another issue with Boolr is the storage of circuits. The circuits are stored in an XML format. In a test build of a small 8-bit machine, the project file quickly bloated to several gigabytes in size. It is not known if this is a fault with the format itself, or simply poor use in the program's implementation. After a point, the file became so large that the program could no longer load it, and doing so would cause the program to lock.

The second piece of software we will look at is Digital, a digital logic simulator with a massive library of features, including a set of analysis tools, such as representing the circuit as a truth table or a Karnaugh map. It is a spiritual successor to Logisim and solved many of the issues associated with it, such as circuit settling to put the circuit in a stable state before simulating it. Digital touts impressive performance capabilities, estimating that a simple processor can be simulated with a 120kHz clock. The developer's description of some core software features will inform some of the design decisions made in this project, specifically the idea of an event based simulator. The developer does not go into detail about what is meant by "event based", but it inspires some attractive ideas about how to approach the problem of ensuring that the simulator's logic is consistent and correct.

Digital and Boolr share some common useful features, the main one being the ability for the user to abstract circuits into black-box circuit packages. This is an invaluable tool in building digital circuits, especially when building something with as many interdependent components as a computer.

Requirements Specification

- Main menu allowing the user to start the simulator in a chosen mode, or exit the program.
  - Lessons : Show list of lessons the user can choose from.
  - Sandbox : Start the simulator in a mode with no objectives.
  - Quit : Exit the program.
- Navigable, grid-based simulator world
  - Movable viewport controlled by the user.
    - Movement achieved via keyboard keys commonly associated with movement.
      - W, A, S and D keys.
      - Arrow keys.
      - Holding Shift should double viewport movement speed.
    - Movement may optionally be implemented via a mouse click-and-drag
  - World should be large enough so as not to restrict the user, even when building very large circuits.
- Circuit packages.
  - Input and output pins for interactions with wires.
  - Adjustable pin positions
  - Name / Symbol
  - Nameable pins (names displayed on body of circuit package)
- Wires
  - Connects circuit package pins and allows transmission of signals between them.
  - Organised in layers (denoted by color).
  - Wires on seperate layers should not connect to each other.
  - Wires should light up when their signal is high.
- Toolbar at bottom of screen, providing a set of common tools the user will need.
  - NOT gate
  - OR gate
  - AND gate
  - XOR gate
  - Three types of wire (red, green and blue).
- Utility buttons at top of screen providing some function unrelated to building circuits.
  - New circuit
    - Open a new world, replacing the current session.
    - Must ask user if they want to save the current session before starting a new one.

    - Open circuit
        - Load existing circuit file in a new session, replacing the current session.
        - Must ask user if they want to save the current session before loading.
    - Save circuit
        - Save the current circuit to a file.
    - New circuit package / integrated circuit
        - Open new session, preserving the current one.
        - Add new utility button allowing user to close the current session and return to the previous session in the stack.
- Button to compact a selected circuit into a user-defined circuit package.
    - Requires input/output nodes that correspond to pins on resulting circuit package.
        - Input/output nodes should be a small, single-pin circuit package.
        - Input nodes' output signal should be manually toggleable by clicking on the node.
        - Both node types should visually indicate their delivered/received signal.
- Button to display a menu of user-created circuit packages.
    - Each circuit must have a name.
    - Clicking on one will close the window and bind a new instance of the selected circuit to the mouse in the same way that clicking on the logic gates in the toolbar would.
- A set of lessons capable of teaching the user how to build a simple computer.
    - Introduction, logic gates and lesson structure
    - Adders
    - Decoders
    - Multiplexers
    - Latches
    - Flip-flops
    - Memory
    - Counters and clocks
    - Arithmetic logic unit
    - General purpose registers
    - Flag registers and the program counter
    - Turing-complete machine

Project Plan

At the time of writing, the project is at a point where the following parts of the requirements specification are complete:

- Navigable, grid-based simulator world
  - Movable viewport controlled by the user.
    - Movement achieved via keyboard keys commonly associated with movement.
      - W, A, S and D keys.
      - Arrow keys.
      - Holding Shift should double viewport movement speed.
  - World is large enough to not restrict the user, even when building very large circuits.
- Wires
  - Connects circuit package pins and allows transmission of signals between them.
  - Organised in layers (denoted by color).
  - Wires on seperate layers should not connect to each other.
- Toolbar at bottom of screen, providing a set of common tools the user will need.
  - NOT gate
  - OR gate
  - AND gate
  - XOR gate
  - Three types of wire (red, green and blue)

The intended course of development will follow this timeline:

- November 2020
  - Circuit packages
- December 2020
  - Logic gates
  - Custom circuit packages
  - Circuit simulation
  - Wire highlighting

- January 2020
  - Custom circuit-packages
  - World serialisation
  - Utility buttons
    - New circuit
    - Save circuit
    - Load circuit
    - New circuit package / integrated circuit
- February 2020
  - Basic lessons
    - Introduction, logic gates and lesson structure
    - Adders
    - Decoders
    - Multiplexers
    - Latches
    - Flip-flops
    - Memory
    - Counters and clocks
- March 2020
  - Final lessons
    - Arithmetic logic unit
    - General purpose registers
    - Flag registers and the program counter
    - Turing-complete machine

Methodology

In a graphical application such as this, where back-end logic must be performant, but also fast to transform into renderable data, there is a demand for a primary focus on the performance of the application. For this reason, the design for this project will focus on how the data itself should be transformed and less on how it is to be represented structurally. With that, the project will be designed with some data-oriented design principles in mind, along with some object-oriented design patterns where appropriate. The reason for this decision is for speed, especially in parts of the program where a lot data will need to be accessed in series, such as tile states, that could be naturally blocked together in a contiguous memory block. Keeping these packaged together and focusing on how the data relates to each other is good way of producing logical groupings of data that benefit the flow of data in the application, as opposed to trying to relate it as closely as possible to the problem domain. Another reason is that it also allows more flexibility in the case of high-level design changes that may otherwise require a lot of change to the implementation. Additionally, circumstances in which certain stages of the pipeline can be processed concurrently will be more clear, and easier to implement.

Implementation of the project will be done in C. The decision for using this language is multi-faceted. For one, it meshes well with the chosen design paradigm. The game engine and tools that will be used in implementing the application are also written in C. There was some consideration put into using C++, however this would make little difference in terms of how the program is implemented, but certain type-checking standards of the language would become intrusive. Implementation will closely follow the design, while employing some techniques and practices that are common in ensuring good performance, robustness and maintainability: branchless programming, pure functions and flattening, to name a few.

Testing, on a more granular scale, will involve unit tests as the software is developed. The structure of the software and the design principles employed will make this process very simple, so long as the associated structures exist, even in a rudimentary sense. As modules are built and connected to each other, the communication of data between them will be evaluated in terms of correctness and performance. Regular checks will be made for memory leaks and performance issues as the software is developed. At a higher level, later in development, the correctness of the logic simulator will be able to be assessed through the lesson system itself. Using the two input/output circuits to supply and verify test data and output to prove that the circuit simulator operates as intended and is fit for purpose.

In evaluating the software, the primary question that needs answering is “does this software teach users how a computer works.” It will take time to get an honest answer to this

question, and will require participation from people who are not particularly well versed in computational logic. It will not be easy for participants to provide this data, as it will require them to complete the lesson series in its entirety, so some level of interest in the subject will be necessary. It appears possible that, if well thought out, the lessons themselves could act as an evaluation of the user's understanding, and completion of the lessons would be proof of the software's effectiveness in teaching the user how a computer works. Some feedback through a light questionnaire could prove useful in verifying that the software achieves its intended goal, and potentially highlight unforseen issues. Subjecting the software to several different users will also help establish its reliability and robustness.

Each stage of development will make use of a set of tools. The most significant are those used in implementation: An in-house game engine, using OpenGL, GLEW, SDL and CGML. Additional tools are used for evaluating the software's memory management and performance: GDB for debugging; Valgrind for tracking memory leaks; and Heaptrack for additional evaluation of memory metrics.

OpenGL: Graphics adapter vendor specific.
GLEW: http://glew.sourceforge.net/
SDL: https://www.libsdl.org/
CGLM: https://github.com/recp/cglm
GDB: https://www.gnu.org/software/gdb/
Valgrind: https://www.valgrind.org/
Heaptrack: https://github.com/KDE/heaptrack